Task 1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

a) Write a C code to create a process using fork() system call.

b) Read the user input as shell command.

c) Use a system call exec() to execute the shell command.

d) Display the PIDs of parent and child processes.

```
shashankreddy@Shashanks-MacBook-Air ~ % nano task2.c
shashankreddy@Shashanks-MacBook-Air ~ %
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    char command[100];

    // Read Linux command
    printf("Enter a Linux command: ");
    scanf("%99s", command);

    // Create child process
    pid_t pid = fork();

    if (pid < 0)
    {
        printf("Fork failed!\n");
        return 1;
    }
    else if (pid == 0)
    {
        // Child process
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        // Execute command
        execlp(command, command, NULL);

        // Executes only if exec fails
        printf("Invalid command!\n");
        exit(1);
    }
    else
    {
        // Parent process
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        // Wait for child
        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

1)Then press ctrl+oenter ctrl+x2)then gcc filename.c -o filename

```
shashankreddy@Shashanks-MacBook-Air ~ % nano task2.c
shashankreddy@Shashanks-MacBook-Air ~ % gcc task2.c -o task2
shashankreddy@Shashanks-MacBook-Air ~ %
```

```
[shashankreddy@Shashanks-MacBook-Air ~ % nano task2.c
[shashankreddy@Shashanks-MacBook-Air ~ % gcc task2.c -o task2
[shashankreddy@Shashanks-MacBook-Air ~ % ./task2
 Enter a Linux command:
```

3)then ./filename

```
[shashankreddy@Shashanks-MacBook-Air ~ % nano task2.c
[shashankreddy@Shashanks-MacBook-Air ~ % gcc task2.c -o task2
[shashankreddy@Shashanks-MacBook-Air ~ % ./task2
 Enter a Linux command: ls

 Parent Process
 Parent PID: 28709
 Child PID: 28712

 Child Process
 Child PID: 28712
 Parent PID: 28709
 6 EXP SIPO &PISO.circ                                                    Music
 680624-radha-krishna-wallpaper-top-free-radha-krishna-background (1).jpg os
 680624-radha-krishna-wallpaper-top-free-radha-krishna-background.jpg     os.c
 7 EXPERIMENT.circ                                                        Pictures
 Applications                                                             Public
 CO_1_RunningTimeCalculation.pdf                                          PyCharmMiscProject
 CO_1_Sorting_Techniques (1).pdf                                          shahank.java
 CO_2_Linked_Lists.pdf                                                    shash.java
 CO_3_Stacks_Queues_Heaps (4).pdf                                         shashank
 CO1___Introduction_to_DS (1).pdf                                         shashank m.c
 Desktop                                                                  shashank.c
 Documents                                                                shashank.java
 Downloads                                                                Shashank.passport
 EXP 7.1.circ                                                             shashankreddy.c
 file                                                                     student.db
 file.c                                                                   task
 file.java                                                                task.c
 first.c.save                                                             task2
 friend_20_problems.xlsx                                                  task2.c
 House-Price-Estimation.pdf                                               vechile
 LearnEnglish-Listening-A1-A-voicemail-message_0.pdf                      vechile.c
 Library                                                                  vechile1.c
 Movies
 Child process completed.
 shashankreddy@Shashanks-MacBook-Air ~ %
```

4)enter the linux command

Child process completed.

Task 2

Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

An operating system is system software that controls all the hardware and software resources of a computer. Users do not communicate directly with the hardware. Instead, the operating system acts as a bridge between the user, applications, and the hardware

Linux Commands Used

1. uname

The uname command is used to display information about the Linux operating system. It provides details such as the operating system name, kernel version, machine architecture, and hostname. This command helps us identify the system on which Linux is running.

```
shashankreddy@Shashanks-MacBook-Air ~ % uname
Darwin
shashankreddy@Shashanks-MacBook-Air ~ %
```

2. lscpu

The lscpu command displays detailed information about the processor. It shows the CPU model, number of cores, number of threads, CPU architecture, and processor speed. This information helps us understand the hardware capabilities of the computer.

```
shashankreddy@Shashanks-MacBook-Air ~ % ./task2
Enter a Linux command: lscpu

Parent Process
Parent PID: 28740
Child PID: 28741

Child Process
Child PID: 28741
Parent PID: 28740
Invalid command!
Child process completed.
shashankreddy@Shashanks-MacBook-Air ~ %
```

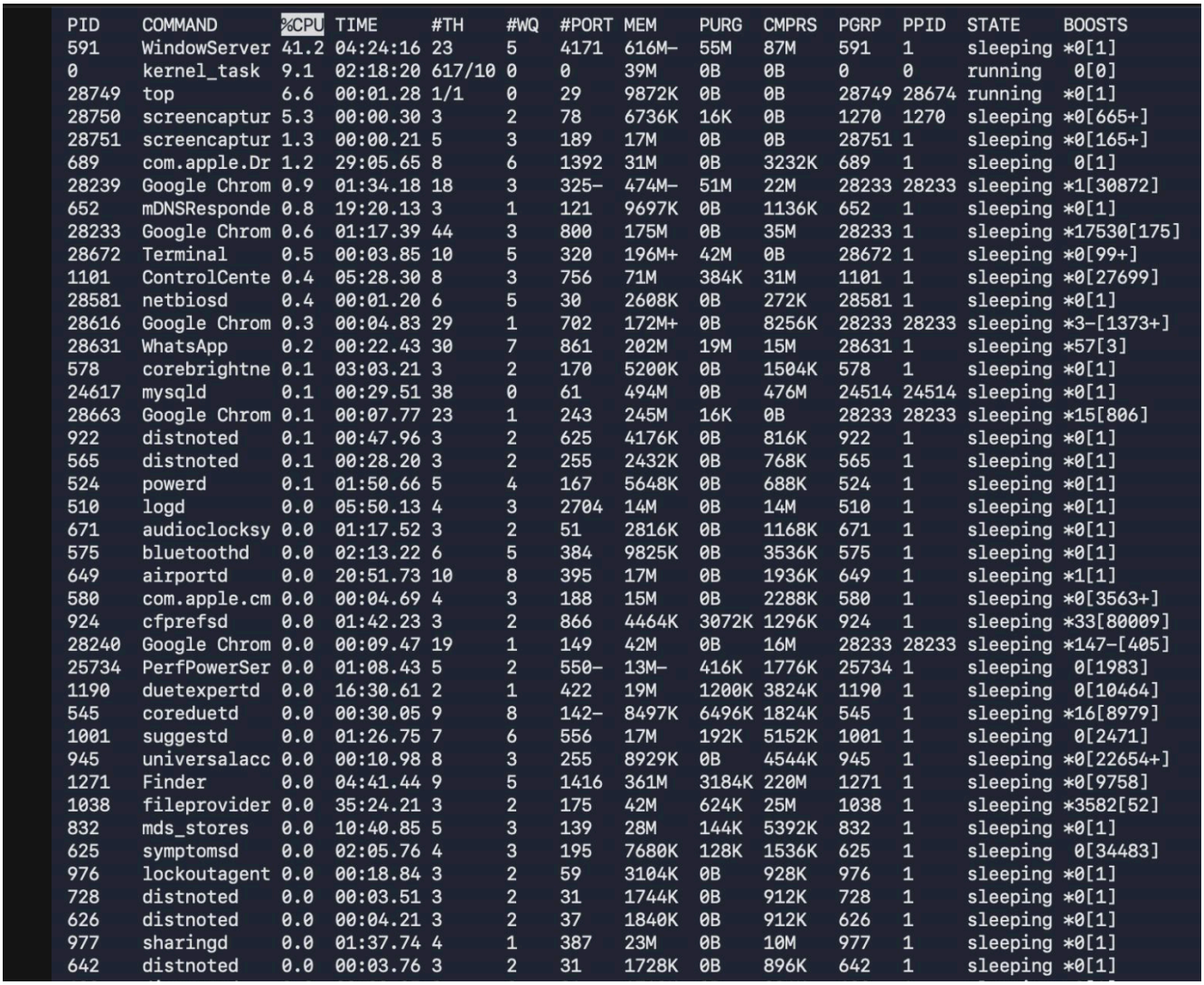
3. ps

The ps command lists the processes that are currently running in the system. It displays the Process ID (PID), Parent Process ID (PPID), user name, and the command associated with each process. This command is useful for monitoring and managing processes.

```
shashankreddy@Shashanks-MacBook-Air ~ % ps
  PID TTY           TIME CMD
28674 ttys000    0:00.05 -zsh
shashankreddy@Shashanks-MacBook-Air ~ %
```

4. top

The top command provides a real-time view of the system's performance. It continuously displays CPU usage, memory usage, running processes, and system load. It is commonly used to identify programs that consume a large amount of system resources.



PID	COMMAND	%CPU	TIME	#TH	#WQ	#PORT	MEM	PURG	CMPRS	PGRP	PPID	STATE	BOOSTS
591	WindowServer	41.2	04:24:16	23	5	4171	616M-	55M	87M	591	1	sleeping	*0[1]
0	kernel_task	9.1	02:18:20	617/10	0	0	39M	0B	0B	0	0	running	0[0]
28749	top	6.6	00:01.28	1/1	0	29	9872K	0B	0B	28749	28674	running	*0[1]
28750	screencaptur	5.3	00:00.30	3	2	78	6736K	16K	0B	1270	1270	sleeping	*0[665+]
28751	screencaptur	1.3	00:00.21	5	3	189	17M	0B	0B	28751	1	sleeping	*0[165+]
689	com.apple.Dr	1.2	29:05.65	8	6	1392	31M	0B	3232K	689	1	sleeping	0[1]
28239	Google Chrom	0.9	01:34.18	18	3	325-	474M-	51M	22M	28233	28233	sleeping	*1[30872]
652	mDNSResponde	0.8	19:20.13	3	1	121	9697K	0B	1136K	652	1	sleeping	*0[1]
28233	Google Chrom	0.6	01:17.39	44	3	800	175M	0B	35M	28233	1	sleeping	*17530[175]
28672	Terminal	0.5	00:03.85	10	5	320	196M+	42M	0B	28672	1	sleeping	*0[99+]
1101	ControlCente	0.4	05:28.30	8	3	756	71M	384K	31M	1101	1	sleeping	*0[27699]
28581	netbiosd	0.4	00:01.20	6	5	30	2608K	0B	272K	28581	1	sleeping	*0[1]
28616	Google Chrom	0.3	00:04.83	29	1	702	172M+	0B	8256K	28233	28233	sleeping	*3-[1373+]
28631	WhatsApp	0.2	00:22.43	30	7	861	202M	19M	15M	28631	1	sleeping	*57[3]
578	corebrightne	0.1	03:03.21	3	2	170	5200K	0B	1504K	578	1	sleeping	*0[1]
24617	mysqld	0.1	00:29.51	38	0	61	494M	0B	476M	24514	24514	sleeping	*0[1]
28663	Google Chrom	0.1	00:07.77	23	1	243	245M	16K	0B	28233	28233	sleeping	*15[806]
922	distnoted	0.1	00:47.96	3	2	625	4176K	0B	816K	922	1	sleeping	*0[1]
565	distnoted	0.1	00:28.20	3	2	255	2432K	0B	768K	565	1	sleeping	*0[1]
524	powerd	0.1	01:50.66	5	4	167	5648K	0B	688K	524	1	sleeping	*0[1]
510	logd	0.0	05:50.13	4	3	2704	14M	0B	14M	510	1	sleeping	*0[1]
671	audioclocksy	0.0	01:17.52	3	2	51	2816K	0B	1168K	671	1	sleeping	*0[1]
575	bluetoothd	0.0	02:13.22	6	5	384	9825K	0B	3536K	575	1	sleeping	*0[1]
649	airportd	0.0	20:51.73	10	8	395	17M	0B	1936K	649	1	sleeping	*1[1]
580	com.apple.cm	0.0	00:04.69	4	3	188	15M	0B	2288K	580	1	sleeping	*0[3563+]
924	cfprefsd	0.0	01:42.23	3	2	866	4464K	3072K	1296K	924	1	sleeping	*33[80009]
28240	Google Chrom	0.0	00:09.47	19	1	149	42M	0B	16M	28233	28233	sleeping	*147-[405]
25734	PerfPowerSer	0.0	01:08.43	5	2	550-	13M-	416K	1776K	25734	1	sleeping	0[1983]
1190	duetexpertd	0.0	16:30.61	2	1	422	19M	1200K	3824K	1190	1	sleeping	0[10464]
545	coreduetd	0.0	00:30.05	9	8	142-	8497K	6496K	1824K	545	1	sleeping	*16[8979]
1001	suggestd	0.0	01:26.75	7	6	556	17M	192K	5152K	1001	1	sleeping	0[2471]
945	universalacc	0.0	00:10.98	8	3	255	8929K	0B	4544K	945	1	sleeping	*0[22654+]
1271	Finder	0.0	04:41.44	9	5	1416	361M	3184K	220M	1271	1	sleeping	*0[9758]
1038	fileprovider	0.0	35:24.21	3	2	175	42M	624K	25M	1038	1	sleeping	*3582[52]
832	mds_stores	0.0	10:40.85	5	3	139	28M	144K	5392K	832	1	sleeping	*0[1]
625	symptomsd	0.0	02:05.76	4	3	195	7680K	128K	1536K	625	1	sleeping	0[34483]
976	lockoutagent	0.0	00:18.84	3	2	59	3104K	0B	928K	976	1	sleeping	*0[1]
728	distnoted	0.0	00:03.51	3	2	31	1744K	0B	912K	728	1	sleeping	*0[1]
626	distnoted	0.0	00:04.21	3	2	37	1840K	0B	912K	626	1	sleeping	*0[1]
977	sharingd	0.0	01:37.74	4	1	387	23M	0B	10M	977	1	sleeping	*0[1]
642	distnoted	0.0	00:03.76	3	2	31	1728K	0B	896K	642	1	sleeping	*0[1]